

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



Học sâu và ứng dụng trong thị giác máy tính (CO5085)

Bài tập

Phân loại ảnh với các mô hình học sâu cơ bản

GVHD: Lê Thành Sách
NHÓM: 12
SVTH: Nguyễn Trung Phong – 2570047



Mục lục

1. Giới thiệu	3
1.1. Tổng quan bài toán phân loại ảnh	3
1.2. Mục tiêu	4
2. Dataset	5
2.1. Dataset sử dụng	5
2.2. Preprocessing	5
3. Phương pháp	7
3.1. Phần 1 — Các mô hình cơ bản	7
3.1.1. Softmax Regression	7
3.1.2. MLP (Multi-Layer Perceptron)	8
3.1.3. SimpleCNN (VGG-style)	9
3.1.4. Vision Transformer (dùng PyTorch built-in)	10
3.2. Phần 2 — Thực nghiệm Huấn luyện, Đánh giá và Đối sánh	11
3.2.1. Training loop	11
3.2.2. Metrics	13
3.2.3. Cấu hình Hyperparameters	13
3.2.4. Kết quả so sánh	14
3.2.5. Nhận xét	15
3.3. Phần 3 — Transformer tự xây	15
3.3.1. Transformer Encoder (custom)	15
3.3.2. Pre-LN TransformerEncoderLayer	16
3.3.3. ViT tự xây	17
3.3.4. So sánh	17
3.4. Phần 4 — Kiến trúc đa dạng	18
3.4.1. CNN + Transformer Hybrid	18
3.4.2. Các cách tokenize ảnh khác nhau	18
3.4.3. So sánh	19
3.5. Phần 5 — LSTM/GRU	20



3.5.1. Biểu diễn ảnh thành chuỗi	20
3.5.2. Kiến trúc	20
3.5.3. So sánh	21
4. Kết luận	22



1. Giới thiệu

1.1. Tổng quan bài toán phân loại ảnh

Phân loại ảnh là bài toán cốt lõi trong lĩnh vực Thị giác máy tính, tập trung vào việc gán nhãn danh mục cho một ảnh đầu vào dựa trên các đặc điểm trực quan của nó. Mục tiêu là xây dựng một ánh xạ toán học từ không gian điểm ảnh sang không gian nhãn lớp tương ứng.

Đầu vào: Tensor định dạng $[H \times W \times C]$ $\rightarrow$ Mô hình dự đoán $\rightarrow$ Đầu ra:
 $\hookrightarrow$ Phân phối xác suất các lớp

Các ứng dụng trong thực tiễn bao gồm: nhận dạng sinh trắc học, chẩn đoán y khoa qua hình ảnh, kiểm soát chất lượng tự động trong công nghiệp, và hệ thống nhận thức trong xe tự hành.

Tiến trình phát triển bài toán phân loại ảnh gắn liền với lịch sử phát triển của mạng lý thuyết học sâu:

Giai đoạn	Khung phương pháp	Đặc điểm cốt lõi
Trước 2012	HOG, SIFT kết hợp SVM	Khai thác đặc trưng thủ công, không tự động tối ưu hóa trong quá trình huấn luyện
2012	AlexNet (Mạng nơ-ron tích chập - CNN)	Đặc trưng được tinh chỉnh thông qua phép tích chập - cột mốc của Kỷ nguyên Deep Learning
2014–2019	VGGNet, ResNet, EfficientNet	Tăng độ sâu kiến trúc kết hợp với thiết kế kết nối thẳng dư
2020–nay	Vision Transformer (ViT)	Tích hợp cơ chế tự chú ý (Self-Attention) từ ngành Xử lý ngôn ngữ tự nhiên



Giai đoạn	Khung phương pháp	Đặc điểm cốt lõi
Hiện tại	Hybrid CNN-Transformer	Khai thác song song tính trừu tượng cục bộ của CNN và khả năng nắm bắt ngữ cảnh toàn cục của Transformer

1.2. Mục tiêu

Báo cáo này tập trung vào việc xây dựng, đánh giá thực nghiệm và phân tích các kiến trúc xử lý phân loại ảnh

Giới hạn	Tên kiến trúc	Đặc điểm mô hình kiến trúc
Phần 1	Softmax Regression, MLP	Khảo sát bài toán tuyến tính và phi tuyến với kết nối đầy đặc (fully-connected)
Phần 1	SimpleCNN, SimpleViT	Đánh giá CNN truyền thống và Vision Transformer rút gọn
Phần 2	Training loop	Thẩm định cơ chế tối ưu hóa động lực học thông qua vòng lặp huấn luyện viết tay
Phần 3	CustomViT	Nghiên cứu sâu về cơ chế Self-Attention từ các phép vi phân và phép nhân ma trận
Phần 4	Multi-modal Tokens, Hybrid CNN-Transformer	Phân tích phương pháp tokenization và kiến trúc lai
Phần 5	LSTM, GRU	Đánh giá khả năng áp dụng mô hình RNN trong xử lý dữ liệu phi không gian

Quy trình huấn luyện được triển khai hoàn toàn từ đầu trên cơ sở dữ liệu **CIFAR-100**, không sử dụng khởi tạo trọng số tinh chỉnh trước (pretrained weights). Training loop được tùy biến thủ công bằng thư viện PyTorch nhằm mục đích hiểu về quá trình backpropagation.



2. Dataset

2.1. Dataset sử dụng

Bài tập này sử dụng **CIFAR-100** (Canadian Institute For Advanced Research, 100 classes).

Bảng 1: Thông tin tập dữ liệu CIFAR-100

Thuộc tính	Giá trị
Tên	CIFAR-100
Số lớp (fine-grained)	100
Số superclass	20
Kích thước ảnh	32×32 pixels, 3 kênh RGB
Tổng số ảnh	60,000
Train / Validation / Test	45,000 / 5,000 / 10,000
Ảnh mỗi lớp (train)	450

Việc lựa chọn CIFAR-100 xuất phát từ nhu cầu thiết lập một bài toán phân loại có độ khó tương xứng. Khác với các mô hình dữ liệu cơ bản, tính chất phức tạp của CIFAR-100 đến từ số lượng nhãn phân loại lên tới 100 lớp, đi kèm với sự khan hiếm tương đối của dữ liệu huấn luyện (chỉ 500 mẫu cho mỗi lớp). Sự phân hóa này tạo ra một “nút thắt cổ chai” về giới hạn khái quát hóa, khiến cho những mô hình tuyến tính đơn giản như Softmax Regression chỉ đạt hiệu suất vô cùng khiêm tốn. Mức baseline cực thấp này vô hình trung tạo ra một thước đo lý tưởng, giúp đánh giá và phản ánh khách quan sức mạnh tối ưu từ đặc trưng cấu trúc (inductive bias) của các kiến trúc nâng cao như CNN hay ViT (Vision Transformer).

2.2. Preprocessing

```
# Xác định trung bình và độ lệch chuẩn của phổ phân phối màu trên CIFAR-100
mean = [0.5071, 0.4867, 0.4408]
std = [0.2675, 0.2565, 0.2761]
```

```
# Kịch bản tăng cường dữ liệu trên Tập Train
train_transforms = transforms.Compose([
    transforms.RandomCrop(32, padding=4), # Phép nội suy ngẫu nhiên với biên
    4 pixels
    transforms.RandomHorizontalFlip(), # Lập nghịch đảo chiều ngang
    transforms.ToTensor(),
    transforms.Normalize(mean, std), # Chuẩn hóa về phân phối thực tiễn
])

# Kịch bản chỉ thực thi chuẩn hóa trên Tập Validation và tập Test
eval_transforms = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(mean, std),
])
```

Listing 1: Data transforms cho CIFAR-100

Tại sao dùng CIFAR-100 stats thay vì ImageNet stats? CIFAR-100 (ảnh 32×32 vật thể) và ImageNet (ảnh tự nhiên lớn) có phân phối màu khác nhau. Dùng sai stats làm dữ liệu bị chuẩn hoá sai, ảnh hưởng đến tốc độ hội tụ và kết quả.

3. Phương pháp

Các mô hình đều tiếp nhận tín hiệu đầu vào là tensor định dạng $[B, 3, 32, 32]$ và xuất ra ma trận dự đoán (logits) $[B, 100]$. Thành phần kích hoạt Softmax không được tính toán trong hàm `forward()` nhằm tối ưu định lý số học, do hàm tổn thất `nn.CrossEntropyLoss` đã ngầm định xử lý qua `log_softmax`.

3.1. Phần 1 — Các mô hình cơ bản

3.1.1. Softmax Regression

Ý tưởng: Là mô hình ánh xạ tuyến tính phi cấu trúc đơn giản nhất. Kiến trúc thực hiện biến đổi trực tiếp từ không gian gradient điểm ảnh thô sang phân phối xác suất lớp bằng một ma trận trọng số đơn đơn lập.

Kiến trúc:

```
Dau vao: [B, 3, 32, 32]
|
v
Flatten -> [B, 3072] (3x32x32 = 3072 features)
|
v
Linear(3072 -> 100) -> [B, 100] logits
```

Code:

```
self.net = nn.Sequential(
    nn.Flatten(),
    nn.Linear(3*32*32, num_classes),
)
```




Bảng 2: Thông số Softmax Regression

Tham số	Giá trị
Input dim	$3 \times 32 \times 32 = 3,072$
Số tham số	$3,072 \times 100 + 100 = \mathbf{307,300}$
Activation	Không (softmax tích hợp trong Loss)
Hạn chế	Chỉ có khả năng cấu trúc các siêu phẳng nội suy tuyến tính, mức độ diễn giải rất thấp với dữ liệu ảnh phức hợp.

3.1.2. MLP (Multi-Layer Perceptron)

Ý tưởng: Thêm các lớp ẩn với hàm kích hoạt phi tuyến ReLU để học được các pattern phức tạp hơn. BatchNorm và Dropout giúp ổn định training và giảm overfitting.

Kiến trúc:

Dau vào: [B, 3, 32, 32]

```
|  
v Flatten -> [B, 3072]  
|  
v Linear(3072->512) -> BN -> ReLU -> Dropout(0.3)  
|  
v Linear(512->256) -> BN -> ReLU -> Dropout(0.3)  
|  
v Linear(256->100) -> [B, 100]
```

Bảng 3: Thông số MLP

Tham số	Giá trị
Lớp ẩn	2 (512 $\rightarrow$ 256 neurons)
Activation	ReLU
Regularization	BatchNorm1d + Dropout(0.3)
Số tham số	$\sim \mathbf{1.7M}$

Hạn chế cấu trúc: Giai đoạn Flatten triệt tiêu hoàn toàn cấu trúc liên kết hạt

nhân trên không gian 2D. Các điểm vị trí kề (i, j) và $(i, j+1)$ có chỉ số quang phân tương đồng mạnh lại bị ánh xạ tĩnh hoàn toàn với nhau. Cấu trúc học phân tán này làm mạng mất khả năng tập hợp ngữ nghĩa cục bộ (local semantic synthesis) — nhược điểm mang tính quyết định mà Mạng Tích chập đã hóa giải thành công.

3.1.3. SimpleCNN (VGG-style)

Ý tưởng: Khai thác cấu trúc không gian 2D của ảnh thông qua tích chập cục bộ. Mỗi filter chỉ “nhìn” vùng 3×3 , học đặc trưng từ vùng lân cận rồi truyền lên lớp cao hơn.

Kiến trúc:

```
Dau vao: [B, 3, 32, 32]
|
v ConvBlock1: [Conv2d(3->32,3x3)+BN+ReLU]x2 -> MaxPool(2) [B,32,16,16]
v ConvBlock2: [Conv2d(32->64,3x3)+BN+ReLU]x2 -> MaxPool(2) [B,64,8,8]
v ConvBlock3: [Conv2d(64->128,3x3)+BN+ReLU]x2 -> MaxPool(2) [B,128,4,4]
v AdaptiveAvgPool2d(1) -> Flatten -> [B, 128]
|
v Linear(128->256) -> ReLU -> Dropout(0.3) -> Linear(256->100)
[B, 100]
```

Bảng 4: Thông số SimpleCNN

Tham số	Giá trị
Conv blocks	3 (channels: $3 \rightarrow 32 \rightarrow 64 \rightarrow 128$)
Kernel size	3×3 , padding=1
Pooling	MaxPool(2) — giảm spatial 1/2 mỗi block
AdaptiveAvgPool	Pool $128 \times 4 \times 4 \rightarrow 128 \times 1 \times 1$
Số tham số	~ 346.6K

Ba lý do CNN vượt trội MLP với dữ liệu ảnh:

1. **Locality:** Mỗi filter chỉ xử lý vùng 3×3 — học đặc trưng cục bộ như cạnh, góc, texture.

2. **Weight sharing:** Cùng bộ filter áp dụng tại mọi vị trí $\rightarrow$ giảm params, tổng quát hoá tốt hơn.
3. **Hierarchical features:** Block 1 học edges, Block 2 học shapes, Block 3 học objects.

3.1.4. Vision Transformer (dùng PyTorch built-in)

Ý tưởng: Thay vì quét nhân tích chập lướt trên hình ảnh, khái niệm Vision Transformer chia ảnh thành các thành tố đa giác nhỏ gọi là “patch” nhằm tạo chuỗi ngữ nghĩa. Từ đó, Transformer đánh giá tương quan giữa các đoạn chuỗi trên thông qua kỹ thuật Self-Attention kế thừa chuẩn từ NLP.

Kiến trúc:

```
Dau vao: [B, 3, 32, 32]
|
v Patch Embedding (Conv2d kernel=4, stride=4)
[B, 128, 8, 8] -> flatten+transpose -> [B, 64, 128]
64 patches, moi patch 4x4 pixels, embed thanh 128-dim vector
|
v Prepend CLS token -> [B, 65, 128]
v + Positional Encoding [1, 65, 128] (learnable)
v TransformerEncoder: 4 layers, d_model=128, nhead=4, ffn=512, Pre-LN
v CLS token -> LayerNorm -> Linear(128->100)
[B, 100]
```

Bảng 5: Thông số SimpleViT

Tham số	Giá trị
Patch size	4×4 ($\rightarrow$ 64 patches trên ảnh 32×32)
d_{model}	128
Số attention heads	4 (mỗi head: 32-dim)
Số Transformer layers	4
FFN dimension	512 ($= 4 \times d_{\text{model}}$)
Positional Encoding	Learnable
LayerNorm	Pre-LN
Số tham số	$\sim 821.0K$

3.2. Phần 2 — Thực nghiệm Huấn luyện, Đánh giá và Đối sánh

3.2.1. Training loop

Theo yêu cầu bài tập, **không dùng** `Lightning trainer.fit()` hay Keras-style API. Tự viết từng bước:

```
def train_one_epoch(model, loader, optimizer, criterion, device,
                    clip_norm=1.0):
    model.train()
    total_loss, total_correct, total_samples = 0, 0, 0

    for x, y in loader:
        x, y = x.to(device), y.to(device)

        # Bước 1: Xóa gradient tu batch trước
        optimizer.zero_grad()

        # Bước 2: Forward pass
        logits = model(x) # [B, 100]
```

```
# Buoc 3: Tinh loss
loss = criterion(logits, y)

# Buoc 4: Backpropagation
loss.backward()

# Buoc 5: Gradient clipping
nn.utils.clip_grad_norm_(model.parameters(), clip_norm)

# Buoc 6: Cap nhat tham so
optimizer.step()

total_loss += loss.item() * x.size(0)
total_correct += (logits.argmax(1) == y).sum().item()
total_samples += x.size(0)

return total_loss / total_samples, total_correct / total_samples
```

Listing 2: Training loop một epoch

Giải thích Gradient Clipping: Nếu norm vector gradient vượt quá $\text{max_norm}=1.0$, scale toàn bộ gradient xuống sao cho $\text{norm} = 1.0$. Kỹ thuật này ngăn “exploding gradients” — đặc biệt quan trọng với RNN và Transformer.

```
def fit(model, train_loader, val_loader, config):
    optimizer = AdamW(model.parameters(), lr=config["lr"],
                       weight_decay=1e-4)
    scheduler = CosineAnnealingLR(optimizer,
                                   T_max=config["epochs"],
                                   eta_min=config["lr"]*0.01)
    best_val_acc, history = 0.0, defaultdict(list)
    for epoch in range(config["epochs"]):
        train_loss, train_acc = train_one_epoch(model, train_loader, ...)
```

```
val_loss, val_acc = evaluate(model, val_loader, ...)
scheduler.step()
if val_acc > best_val_acc:
    best_val_acc = val_acc
    torch.save(model.state_dict(), config["save_path"])
return history
```

Listing 3: Vòng lặp huấn luyện fit()

3.2.2. Metrics

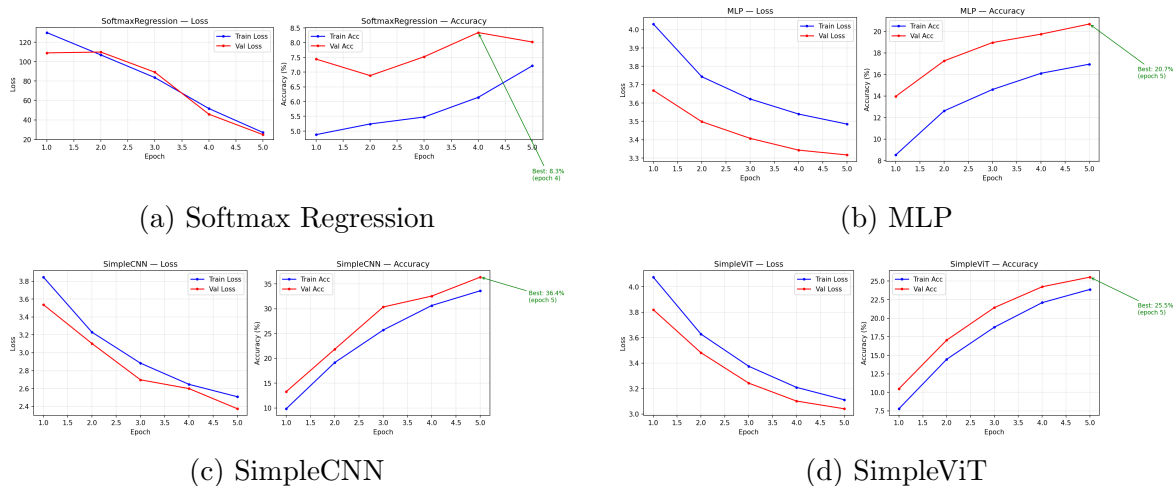
- **Accuracy:** % ảnh phân loại đúng trên tập test.
- **F1-macro:** Trung bình F1 của 100 lớp — đánh giá đều trên tất cả lớp, không bị lớp lớn lấn át.
- **Loss:** CrossEntropyLoss — đo độ tin cậy của xác suất dự đoán.

3.2.3. Cấu hình Hyperparameters

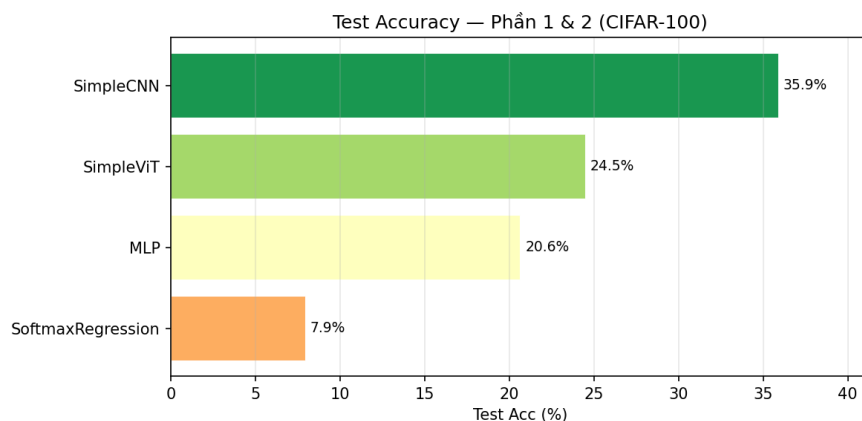
Bảng 6: Hyperparameters Phần 1+2

Mô hình	LR	Batch	Epochs	Optimizer	Scheduler
SoftmaxRegression	0.1	256	30	AdamW	CosineAnnealing
MLP	1e-3	128	50	AdamW	CosineAnnealing
SimpleCNN	1e-3	128	50	AdamW	CosineAnnealing
SimpleViT	3e-4	128	100	AdamW	CosineAnnealing

3.2.4. Kết quả so sánh



Hình 1: Training curves của 4 mô hình Phần 1



Hình 2: So sánh Test Accuracy 4 mô hình Phần 1+2. SimpleCNN dẫn đầu với 35.88%, gấp hơn 4× Softmax Regression.

Bảng 7: Kết quả 4 mô hình Phần 1+2 trên CIFAR-100 test set

Mô hình	Test Acc	Val Acc	F1-macro	Params	Acc/Param
SoftmaxRegression	7.94%	8.34%	6.09%	307.3K	25.8%/M
MLP	20.63%	20.68%	18.31%	1.7M	12.1%/M
SimpleViT	24.47%	25.54%	22.35%	821.0K	29.8%/M
SimpleCNN	35.88%	36.38%	34.48%	346.6K	103.5%/M

3.2.5. Nhận xét

Softmax Regression (7.94%): Chỉ học được biên quyết định tuyến tính trong không gian 3072-chiều — chỉ tốt hơn random (1%) khoảng 8 lần. F1-macro (6.09%) thấp hơn accuracy (7.94%) cho thấy mô hình tập trung vào vài lớp dễ, bỏ qua phần lớn lớp còn lại.

MLP (20.63%): Các lớp ẩn với ReLU học được đặc trưng phi tuyến, cải thiện $2.6\times$ so với Softmax. Tuy nhiên Flatten phá vỡ quan hệ không gian 2D: hai pixels kề nhau bị xử lý như hai features độc lập.

SimpleCNN (35.88%) — tốt nhất Phần 1: CNN (346.6K params) vượt MLP (1.7M params) — bằng chứng **inductive bias phù hợp quan trọng hơn số tham số**. Acc/Param = 103.5%/M — hiệu quả tốt nhất trong 4 mô hình.

SimpleViT (24.47%): Thấp hơn CNN vì thiếu spatial inductive bias. Với 45K mẫu (450/lớp), Transformer chưa đủ data để vượt CNN. Val accuracy vẫn tăng ở epoch 100 $\rightarrow$ cần nhiều epochs hơn.

3.3. Phần 3 — Transformer tự xây

3.3.1. Transformer Encoder (custom)

Hiện thực TransformerEncoder từ các phép toán cơ bản nhằm làm rõ cơ chế hoạt động nội tại của self-attention, thay vì sử dụng các API mức cao được đóng gói sẵn.

Lý thuyết Scaled Dot-Product Attention:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V \quad (1)$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_{\text{head}}}}\right)V \quad (2)$$

Chia $d_{\text{model}} = 128$ thành $H = 4$ heads, mỗi head có $d_{\text{head}} = 32$.

Multi-Head Attention bằng torch.einsum:

```
class CustomMultiHeadAttention(nn.Module):  
    def __init__(self, d_model, num_heads, dropout=0.1):
```



```
self.W_q = nn.Linear(d_model, d_model, bias=False)
self.W_k = nn.Linear(d_model, d_model, bias=False)
self.W_v = nn.Linear(d_model, d_model, bias=False)
self.W_o = nn.Linear(d_model, d_model)
self.d_head = d_model // num_heads

def forward(self, x): # x: [B, T, d_model]
    B, T, _ = x.shape
    H, d_h = self.num_heads, self.d_head

    Q = self.W_q(x).reshape(B,T,H,d_h).transpose(1,2)
    K = self.W_k(x).reshape(B,T,H,d_h).transpose(1,2)
    V = self.W_v(x).reshape(B,T,H,d_h).transpose(1,2)

    # 'bhjd,bhjd->bhij': batch, head, query_pos, key_pos, d_head
    scores = torch.einsum('bhjd,bhjd->bhij', Q, K) / math.sqrt(d_h)
    attn = F.softmax(scores, dim=-1)

    out = torch.einsum('bhij,bhjd->bhid', attn, V)
    out = out.transpose(1,2).reshape(B, T, self.d_model)
    return self.W_o(out)
```

Listing 4: Custom Multi-Head Attention

3.3.2. Pre-LN TransformerEncoderLayer

```
class CustomTransformerEncoderLayer(nn.Module):
    def __init__(self, d_model, num_heads, ffn_dim=None, dropout=0.1):
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.attn = CustomMultiHeadAttention(d_model, num_heads, dropout)
        self.ffn = nn.Sequential(
            nn.Linear(d_model, ffn_dim),
```

```

        nn.GELU(),
        nn.Dropout(dropout),
        nn.Linear(ffn_dim, d_model),
    )

    def forward(self, x):
        x = x + self.attn(self.norm1(x)) # Pre-LN residual
        x = x + self.ffn(self.norm2(x))
        return x

```

Listing 5: Custom TransformerEncoderLayer (Pre-LN)

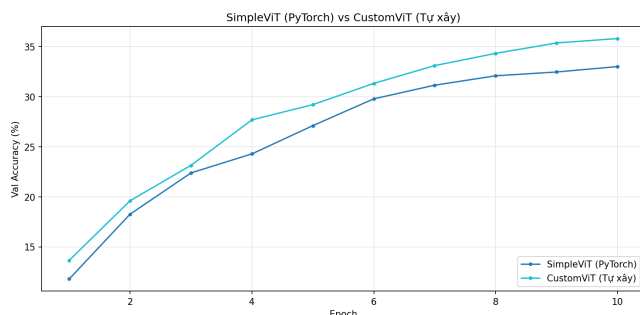
Bảng 8: Pre-LN vs Post-LN

Variant	Công thức	Gradient	Warmup
Post-LN (Vaswani 2017)	$x = \text{LN}(x + \text{Attn}(x))$	Không ổn định	Có
Pre-LN (được chọn)	$x = x + \text{Attn}(\text{LN}(x))$	Ổn định từ đầu	Không

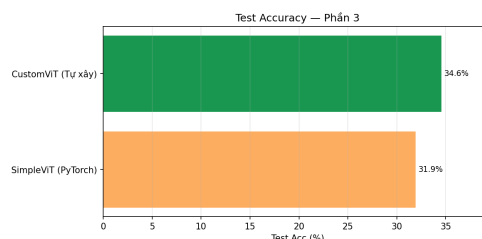
3.3.3. ViT tự xây

Giống hệt SimpleViT (Phần 1) — cùng patch embedding, CLS token, positional encoding, classification head **chỉ thay** `nn.TransformerEncoder` bằng `CustomTransformerEncoder`.

3.3.4. So sánh



(a) Training curves



(b) Test Accuracy

Hình 3: So sánh SimpleViT (PyTorch) và CustomViT (tự xây)



Bảng 9: Kết quả Phần 3

Mô hình	Encoder	Test Acc	Val Acc	F1	Params
SimpleViT (PyTorch)	nn.TransformerEncoder	31.92%	33.00%	30.39%	821.0K
CustomViT	Custom einsum	34.56%	35.80%	33.11%	819.4K

Nhận xét: Training curves tương tự $\rightarrow$ xác nhận Custom MHA hoạt động đúng về mặt toán học. Chênh lệch 2.64% nằm trong biên độ variance ngẫu nhiên — **kết luận:** hai mô hình tương đương nhau.

3.4. Phần 4 — Kiến trúc đa dạng

Ba cách tokenize ảnh khác nhau được triển khai và so sánh.

3.4.1. CNN + Transformer Hybrid

Ý tưởng: Dùng CNN làm backbone trích đặc trưng tạo tokens chất lượng cao, sau đó đưa vào Transformer để học quan hệ toàn cục.

```
Dau vao: [B, 3, 32, 32]
|
v CNN Backbone
ConvBlock1: Conv(3->32)+BN+ReLU+Pool -> [B, 32, 16, 16]
ConvBlock2: Conv(32->64)+BN+ReLU+Pool -> [B, 64, 8, 8]
|
v Tokenize: Reshape -> [B, 64, 64]
v Linear(64->128) -> [B, 64, 128] 64 high-quality tokens
|
v CLS + PosEmbed + 2 x TransformerLayer(d=128, h=4)
v CLS token -> Linear(128->100)      Params: 446.1K
```

3.4.2. Các cách tokenize ảnh khác nhau

SpatialToken ViT — coi $H \times W$ positions làm tokens:

```
[B, 3, 32, 32] -> Reshape -> [B, 1024, 3]
```

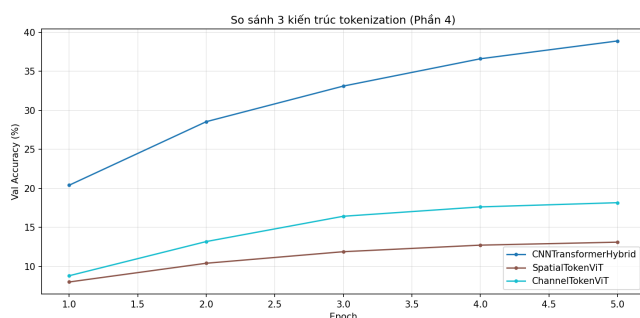


```
Linear(3->64) -> [B, 1024, 64]
PosEmbed + 2 x TransformerLayer(d=64, h=4)
GlobalAvgPool -> Linear(64->100)      Params: 172.4K
[!] Attention: [B, 4, 1024, 1024] ~512MB RAM -> batch_size=32
```

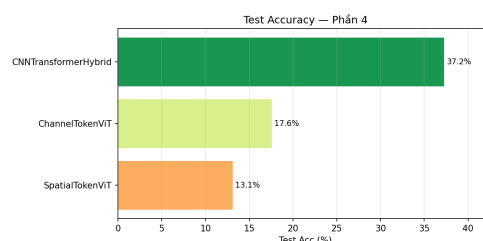
ChannelToken ViT — coi C channels làm tokens:

```
[B, 3, 32, 32]
Conv2d(3->64, 1x1)+BN+ReLU -> [B, 64, 32, 32]
Reshape -> [B, 64, 1024]
Linear(1024->128) -> [B, 64, 128]
PosEmbed + 2 x TransformerLayer(d=128, h=4)
GlobalAvgPool -> Linear(128->100)      Params: 549.5K
```

3.4.3. So sánh



(a) Training curves



(b) Test Accuracy

Hình 4: So sánh 3 kiến trúc Phần 4

Bảng 10: Kết quả Phần 4

Mô hình	Tokenize	Seq	Test Acc	Val Acc	F1	Params
CNN+Transformer	CNN features	64	37.25%	38.90%	35.87%	446.1K
ChannelToken ViT	C channels	64	17.56%	18.18%	15.29%	549.5K
SpatialToken ViT	H×W pixels	1,024	13.10%	13.12%	9.86%	172.4K

Nhận xét: CNN+Transformer (37.25%) — tốt nhất toàn bài. SpatialToken ViT thất bại vì 3 RGB/token quá thô. **Kết luận:** Token quality > Token quantity —



SpatialToken có $16\times$ nhiều tokens hơn nhưng kết quả tệ hơn $2.8\times$.

3.5. Phần 5 — LSTM/GRU

3.5.1. Biểu diễn ảnh thành chuỗi

Bảng 11: Các cách biểu diễn ảnh thành sequence

Seq Mode	T	D	Cách biểu diễn
Row-wise	32	$96 (= 32 \times 3)$	Mỗi hàng pixels
Col-wise	32	$96 (= 32 \times 3)$	Mỗi cột pixels
Patch4	$64 (= 8 \times 8)$	$48 (= 4 \times 4 \times 3)$	64 patches 4×4
Patch8	$16 (= 4 \times 4)$	$192 (= 8 \times 8 \times 3)$	16 patches 8×8

3.5.2. Kiến trúc

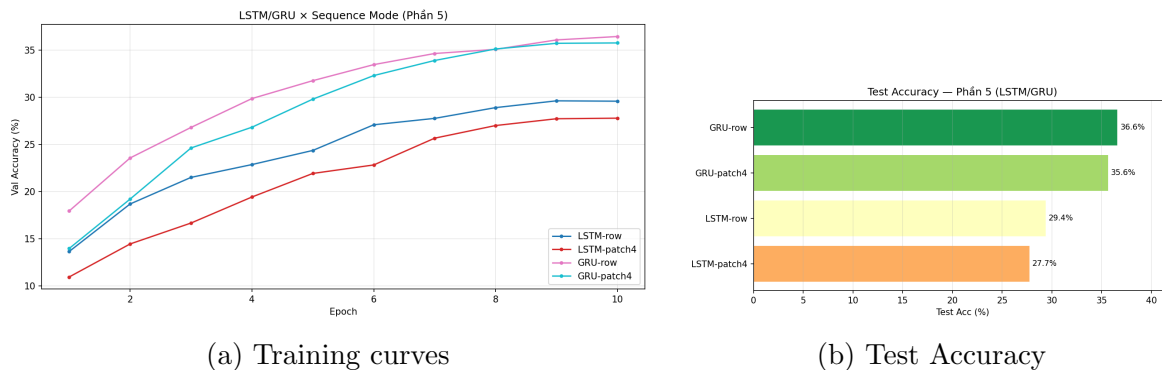
Dau vao: [B, T, D]

```
|  
v BiLSTM / BiGRU (hidden=256, num_layers=2, bidirectional=True,  
                  dropout=0.3)  
v concat(h_n[-2], h_n[-1]) -> [B, 512]  
v Dropout(0.3) -> Linear(512->100) -> [B, 100]
```

Bảng 12: So sánh LSTM vs GRU

Đặc điểm	LSTM	GRU
Số gates	4: forget, input, output, cell	2: reset, update
Cell state	Có (c_t)	Không
Params/layer	$4(D + H)H + 4H$	$3(D + H)H + 3H$
Phù hợp	$T > 1000$ steps	$T \leq 100$ steps

3.5.3. So sánh



Hình 5: So sánh LSTM và GRU với các cách biểu diễn chuỗi

Bảng 13: Kết quả Phần 5

Mô hình	Mode	T	Test Acc	Val Acc	F1	Params
GRU	Row	32	36.57%	36.44%	35.68%	1.8M
GRU	Patch4	64	35.64%	35.76%	34.99%	1.7M
LSTM	Row	32	29.36%	29.62%	28.34%	2.4M
LSTM	Patch4	64	27.73%	27.78%	26.41%	2.3M

Nhận xét: GRU vượt LSTM +7.2% (row) vì chuỗi ngắn $T = 32$ không cần cell state riêng biệt của LSTM — GRU ít params hơn → ít overfitting hơn. GRU-row (36.57%) cạnh tranh ngang SimpleCNN (35.88%) nhưng cần $5\times$ nhiều tham số hơn.

4. Kết luận

Dựa trên các thực nghiệm, báo cáo đã rút ra những kết luận nền tảng về của các kiến trúc học sâu phân loại hình ảnh. Trước tiên, bản chất của Inductive Bias đóng vai trò quyết định đối với năng lực khái quát hóa tổng thể. Thay vì đơn thuần mở rộng không gian cực hạn tham số như mô hình MLP (1.7M) hay LSTM (2.4M), thiết kế Mạng Tích chập SimpleCNN mang lại hiệu suất vượt bậc chỉ với một lượng thông số nhỏ gọn (346.6K). Điều này minh chứng rõ nét rằng việc kiến trúc mô hình đồng nhất với tính chất spatial topology đóng giá trị rất lớn vào quy trình tối ưu trọng số. Bên cạnh đó, quy mô dữ liệu huấn luyện thể hiện mối tương quan ràng buộc ngặt nghèo đối với kiến trúc ViT. Khi bị đóng khung trong tập 45,000 mẫu hình ảnh chưa được trained, hiệu năng dao động của ViT (24–34%) hoàn toàn lép vế trước CNN (36–37%). Rõ ràng, bản thể mô hình Transformer yêu cầu lượng tri thức tiên nghiệm vĩ mô, và sức mạnh đặc thù đó thường chỉ được khai phóng khi sử dụng pretrained weight từ siêu cơ sở dữ liệu quy mô hàng chục triệu ảnh (như ImageNet-21K).

Bằng cách kết nối cục bộ mạnh mẽ của CNN với hệ quy chiếu tương quan toàn cục của cơ chế Tự Chú ý (Self-Attention), kiến trúc lai CNN+Transformer xác lập mốc đỉnh cao hiệu năng nhất toàn bộ báo cáo (37.25%). Không dừng lại ở luồng kiến trúc, nghiên cứu cũng chỉ điểm nghịch lý cốt lõi về biểu diễn định tính và định lượng của tập token. Cuối cùng, việc kiểm tra độ tương thích chuỗi tĩnh giới hạn cho thấy kết cấu tối giản cổng lập tích hợp tinh gọn của biến thể GRU chặn đứng hiệu quả chứng lạm học (overfitting) do tham số, nhờ đó đẩy lùi triệt để giới hạn dự báo tồn đọng trên mạng bộ nhớ cổng kênh LSTM.